

Lecture 10: Digitisation

LSE ME314: Introduction to Data Science and Machine Learning (<https://github.com/me314-lse>)

2026-07-28

Ryan Hübert

What is digitisation?

From the world to a table

Data starts as experiences: sounds, images, language, places.

To analyse it with a computer, it has to make two trips:

1. **Digitisation**: get it into the computer, as **bits**
2. **Structuring**: turn those bits into a table we can analyse

Both trips lose information and involve choices.

Depending on the type of data, one of the trips can be harder than the other.

Key: a table is something we *build*, not something we are handed.

Which trip is the hard one?

The hard trip depends on the data:

- Sound, images, and space are all **analog**.
 - Analog data is continuous data.
 - Digitisation is the hard part.
 - But once digitised, they are already an array.
- Text is **discrete**.
 - Getting it in is easy, since it is (nowadays) already digital.
 - Structuring into tabular data is the hard part.

So each lecture tackles a different trip:

- Part 1 (today): trip 1, getting each type into bits.
- Part 2: trip 2, turning text into a table.

What is digital data?

Trip 1 turns experiences into **digital data**.

→ Why? Computers can only work with digital data.

Digital data represents information as bits: 0s and 1s.

→ With n bits, can represent 2^n patterns

→ E.g., 2 bits can represent four possibilities: 00, 01, 10, 11

→ 2 bits are needed to represent a cardinal direction (N, S, E, W)

Bytes are an aggregation of bits: 8 bits = 1 byte.

→ kilobytes (KB), megabytes (MB), gigabytes (GB), etc. are metric aggregations of bytes (**roughly**)

→ $\neq$ kilobits (Kb), megabits (Mb), gigabits (Gb), etc.

For technical reasons: smallest unit on computer is a byte (not bit).

Bytes in binary, decimal and hex

A single byte can represent up to $2^8 = 256$ pieces of information.

There are several conventions used to write a byte.

1. **Binary** (**bin** or **base 2**) directly writes bits, e.g. 00101010.
2. **Decimal** (**dec** or **base 10**) assigns a distinct integer from 0-255 to each possible combination of bits, e.g., 42.
3. **Hexadecimal** (**hex** or **base 16**) writes each 4 bits as one of sixteen symbols (0-9, then a-f), e.g., 2a.
 - We refer to 2a as a **hex code**.
 - Hex codes are the most commonly used format for writing bytes.
 - Sometimes, hex codes are preceded with 0x or \x so that it's clear you are looking at a hex code, e.g. \x2a or 0x2a.
 - R uses hex codes as its default for writing bytes (as you'll see).
 - Read more about hex codes [here](#).

An object in R is bytes

Every object you make in R is stored as bits and bytes.

R prints it in a readable form:

```
x <- 42L # an integer object  
print(x) # print it
```

```
[1] 42
```

But you can ask for the byte behind it, in hex, decimal or binary:

```
as.raw(x) # hex
```

```
[1] 2a
```

```
as.integer(x) # dec
```

```
[1] 42
```

```
paste(rev(as.integer(rawToBits(as.raw(x)))), collapse = "") # bin
```

```
[1] "00101010"
```

That isn't the memory layout

Those show the value 42.

In memory the integer actually takes four bytes:

```
writeBin(x, raw())
```

```
[1] 2a 00 00 00
```

The same number as a double is eight different bytes:

```
writeBin(42, raw())
```

```
[1] 00 00 00 00 00 00 45 40
```

Files are bytes too

Files on your computer are bytes too.

Reading a file translates those bytes into objects you recognise:

```
library("readr")  
df <- read_csv("people.csv") # comma-delimited  
head(df) # first six rows
```

```
# A tibble: 3 x 4  
  name    age city    height_m  
  <chr> <dbl> <chr>    <dbl>  
1 Ada      34 London    1.71  
2 Bela     29 Berlin    1.8  
3 Chen     45 Shanghai  1.63
```

Reading the raw bytes

You can also read the bytes with no translation:

```
df.raw <- read_file_raw("people.csv")  
head(df.raw) # first six bytes (not rows)
```

```
[1] 6e 61 6d 65 2c 61
```

Then convert them yourself, into text or bits:

```
rawToChar(head(df.raw, 22)) # bytes -> text (the header row)
```

```
[1] "name,age,city,height_m"
```

```
sapply(head(df.raw), function(b)  
  paste(rev(as.integer(rawToBits(b))), collapse = "")) # bytes -> bits
```

```
[1] "01101110" "01100001" "01101101" "01100101" "00101100"
```

```
[6] "01100001"
```

A file's size is its bytes

A file's size is exactly its number of bytes:

```
length(df.raw)
```

```
[1] 84
```

In the computer file browser, check that the file is the same number of bytes.

people.csv

comma-separated values - 84 bytes

(Windows computers might vary slightly)

Encoding: rules for representing data

Human-readable or machine-readable?

We just read a file as raw bytes: 6e 61 6d 65 ... (hex) or 01101110 ... (bits).

- Could you tell it was a table of people? Neither could any human.

All digital data is bits and bytes, so all of it is **machine-readable**.

Most of it is *not* **human-readable**:

- Human-readable data is text (open it in a text editor and you see text)
- Everything else is loosely called **binary** (an image, an audio file)

“Readable” is narrow here: *is it text?*, not *do you understand it?*

So how does 6e 61 6d 65 become the word “name” on screen?

What is encoding?

A byte pattern has no intrinsic meaning without a *rule* for reading it.

An **encoding** is a rule for representing data in another form.
Decoding reads it back.

- Both sides must share the rule, or you get garbage.
- **Lossless** encodings recover the original exactly.
- **Lossy** encodings throw data away to save space.

Two encoding tasks we'll cover:

- **Character encoding**: represent text as bytes
- **Base64**: represent binary bytes as text

Bytes don't say what they are

The same bytes support many readings.

For example, the bytes 6e 61 6d 65 could be:

- the text “name”
- four integers (110, 97, 109, 101)
- part of an image, a colour, a sound...

Nothing in the bytes says which, so any software reading them must assume a rule.

- It infers one from the file extension (.jpg), the first “magic” bytes (ff d8 = JPEG), or metadata
- Right assumption → your data. Wrong assumption → garbage.

Characters and strings

Our first task, character encoding: turn text into bytes. Text is made of characters.

A **character** is the “smallest component of written language that has semantic value”

→ See <https://unicode.org/glossary/#character>

→ For example, “A”, “ü”, “2”, etc.

A **(character) string** is a sequence of characters

→ Strings are at the core of all analysis of digital texts

→ R refers to strings as “character” objects, which is somewhat imprecise

Character encoding

A character encoding defines how a machine “translates” bits/bytes of data into strings

A character encoding consists of a **character set**: a list of characters with associated numerical representations

- **Code points** are the unique “numbers” associated with characters in a character set
- These can be expressed in multiple formats
- The code point is the character's ID; the encoding decides which bytes represent it

Encodings tell the computer how to render text when given bytes

- There are different encodings, using differing numbers of bits per character (7-bit, 8-bit, 16-bit, etc.)

The origins of encoding: ASCII

ASCII: the original character set/encoding, uses 7 bits

→ Could only encode up to $2^7 = 128$ characters... not enough!

ASCII was later extended to 8 bits, e.g. **ISO-8859-1**

→ Now could encode $2^8 = 256$ characters... still not enough!

Example: hex code point for “A” is 41 with bits: 1000001 (original)
01000001 (extended)

As you can imagine: different languages, different characters →
different character sets and encodings

A mess... see http://en.wikipedia.org/wiki/Character_encoding

The origins of encoding: ASCII

Each character is one number (its **code point**), which we can write four ways:

Character	Binary	Decimal	Hex
(space)	00100000	32	20
0	00110000	48	30
A	01000001	65	41
B	01000010	66	42
a	01100001	97	61
b	01100010	98	62

Consecutive letters get consecutive numbers; lower-case sits 32 above upper-case.

Widely used character encoding today: Unicode

Unicode is an encoding standard that is now commonly used

→ Has several encoding formats, e.g. **UTF-8**, **UTF-16**, **UTF-32**

UTFs differ by how many bytes they use:

- UTF-8 is a **variable-width encoding**: 1, 2, 3 or 4 bytes
- UTF-16 mostly uses 2 bytes, sometimes 4 bytes
- UTF-32 is a **fixed-width encoding** using 4 bytes
- UTF-16 and UTF-32 involve **null bytes** (00000000)

UTF-8 is now the default character encoding for most digital data

- Accommodates a vast number of characters very efficiently

UTF-8 examples

	Code point	Byte 1	Byte 2	Byte 3	Byte 4
&	U+0026	00100110			
u	U+0075	01110101			
ü	U+00FC	11000011	10111100		
Д	U+0414	11010000	10010100		
Λ	U+120A	11100001	10001000	10001010	
😊	U+1FAE0	11110000	10011111	10101011	10100000

UTF-16 examples

	Code point	Byte 1	Byte 2	Byte 3	Byte 4
&	U+0026	00100110	00000000		
u	U+0075	01110101	00000000		
ü	U+00FC	11111100	00000000		
Д	U+0414	00010100	00000100		
Λ	U+120A	00001010	00010010		
😊	U+1FAE0	00111110	11011000	11100000	11011110

Note: these are **little endian**, where null bytes go last (instead of **big endian**)

UTF-32 examples

	Code point	Byte 1	Byte 2	Byte 3	Byte 4
&	U+0026	00100110	00000000	00000000	00000000
u	U+0075	01110101	00000000	00000000	00000000
ü	U+00FC	11111100	00000000	00000000	00000000
Д	U+0414	00010100	00000100	00000000	00000000
Λ	U+120A	00001010	00010010	00000000	00000000
😄	U+1FAE0	11100000	11111010	00000001	00000000

Note: these are little endian, where null bytes go last (instead of big endian)

Encoding choice changes file size

Because a file's size is its number of bytes, the encoding changes the size. The same text:

```
text <- "Ryan Hübert"  
length(charToRaw(text)) # UTF-8 bytes (
```

```
[1] 12
```

```
length(iconv(text, "UTF-8", "UTF-32LE", toRaw = TRUE)[[1]]) # UTF-32 b
```

```
[1] 44
```

- UTF-8: 1 byte per ASCII character, more only when needed
- UTF-32: always 4 bytes
- So for mostly-English text, UTF-8 files are far smaller

Character encoding in R

Newer versions of R and Positron (and most modern software) use UTF-8 as their default character encoding

So, any digital text data created on the fly is UTF-8 encoded

→ All text displayed in the console or the editor panel is also UTF-8 encoded

You can verify this by looking at the raw bytes:

```
charToRaw("ü") # this demonstrates UTF-8 encoding
```

```
[1] c3 bc
```

Two bytes (c3 bc) for one character: that is variable-width UTF-8 at work.

Encoding issues: mojibake

Software must *guess* a rule to read bytes.

For text, the guess often fails:

- a text file usually doesn't record its own encoding
- guess wrong and the characters come out wrong
- this is called **mojibake**

Watch out for:

- MS Office (Windows-1252),
- Windows (UTF-16) vs Unix (UTF-8), and
- editors that *display* mojibake even when the file is fine.

Mojibake in the wild

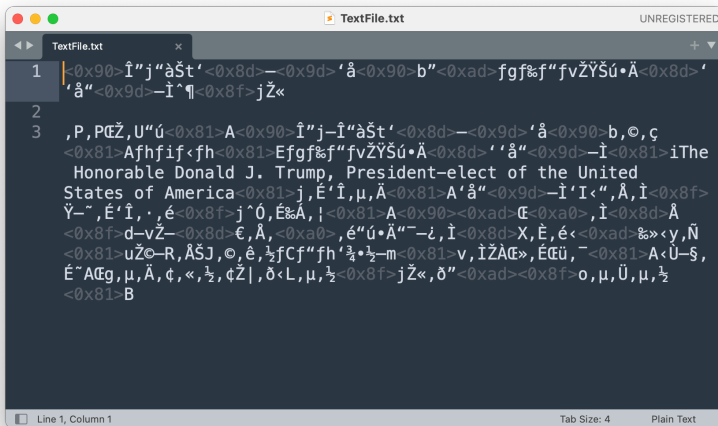
We can read a text file directly into strings with `read_file()`:

```
unk <- read_file("TextFile.txt")  
print(unk)
```

```
[1] "\x83e\x83L\x83X\x83g\x83t\x83@\x83C\x83\x8b"
```

This looks like mojibake: R has assumed the wrong encoding.

Mojibake in the wild



The screenshot shows a text editor window titled "TextFile.txt" with a status bar indicating "UNREGISTERED". The editor displays three lines of text. Line 1 contains a string of mojibake characters. Line 2 is empty. Line 3 contains a mix of mojibake and the beginning of a sentence in English: ", P, PĚŽ, U“ú<0x81>A<0x90>Î”j-Î“àŠt‘<0x8d>-<0x9d>‘â<0x90>b,©,ç". The status bar at the bottom shows "Line 1, Column 1", "Tab Size: 4", and "Plain Text".

```
1 <0x90>Î”j-Î“àŠt‘<0x8d>-<0x9d>‘â<0x90>b”<0xad>fgf%”fvŽŸŠú•Ă<0x8d> ‘
   ‘â”<0x9d>-Î^¶<0x8f>jŽ«
2
3 , P, PĚŽ, U“ú<0x81>A<0x90>Î”j-Î“àŠt‘<0x8d>-<0x9d>‘â<0x90>b,©,ç
   <0x81>Afhfif<fh<0x81>Efgf%”fvŽŸŠú•Ă<0x8d>‘ ‘â”<0x9d>-Î<0x81>iThe
   Honorable Donald J. Trump, President-elect of the United
   States of America<0x81>j,É’Î,μ,Ă<0x81>A’â”<0x9d>-Î’I“,Ă,İ<0x8f>
   Ÿ~,É’Î,.,é<0x8f>j^Ó,É%Á,|<0x81>A<0x90><0xad>Œ<0xa0>,İ<0x8d>Ă
   <0x8f>d-vŽ-<0x8d>€„Ă,<0xa0>,é“ú•Ă”-i,İ<0x8d>X,È,é<0xad>%»<y,Ŋ
   <0x81>uŽ©-R,ÅŠJ,©,ê,½fCf”fh’¼•½-m<0x81>v,İŽĂŒ»,ÉĚü,-<0x81>A<Ü-š,
   É~AŒg,μ,Ă,¢,«,½,¢Ž|,ð<L,μ,½<0x8f>jŽ«,ð”<0xad><0x8f>o,μ,Ŭ,μ,½
   <0x81>B
```


Fixing mojibake: guess the encoding

First, ask R to try to guess the encoding:

```
guess_encoding(charToRaw(unk))
```

```
# A tibble: 3 × 2
  encoding confidence
  <chr>         <dbl>
1 SHIFT_JIS      0.99
2 ISO-8859-1     0.35
3 ISO-8859-9     0.24
```

Here we see it is pretty confident, but it isn't always.

Fixing mojibake: decode and re-save

If you know the encoding (or R's guess seems promising), you can “decode” the character object:

```
x <- parse_character(unk, locale = locale(encoding = "Shift-JIS"))  
print(x)
```

[1] "石破内閣総理大臣発トランプ次期米国大統領宛祝辞\n\n1 1月6日、石破茂内閣総理大臣から、ドナルド・トランプ次期米国大統領（The Honorable Donald J. Trump, President-elect of the United States of America）に対して、大統領選挙での勝利に対する祝意に加え、政権の最重要事項である日米同盟の更なる強化及び「自由で開かれたインド太平洋」の実現に向け、緊密に連携していきたい旨を記した祝辞を発出しました。"

When satisfied, save the resulting text as UTF-8 for the future:

```
write_file(x, "TextFileUTF8.txt")
```

A lot of tabular data is stored as text

A .csv file is just text: rows of characters, with commas marking the columns.

```
read_file("people.csv") # a CSV is just lines of text
```

```
[1] "name,age,city,height_m\nAda,34,London,1.71\nBela,29,Berlin,1.80\nC
```

`read_csv()` parses that text into a table:

- splits each line on ,
- reads the header row
- infers a **type** per column (numeric, character, ...)

When bytes aren't text at all

Character encoding handles text. But lots of data is binary: non-text, like an image or audio file.

A .jpg image is binary: its raw bytes (ff d8 ff e0 ...) are *not* text.

```
ryan.jpg.file <- "ryan_hubert.jpg"    # a photo on disk
read_file(ryan.jpg.file)              # not human readable!
read_file_raw(ryan.jpg.file)[1:15]    # raw bytes (first 15)
```

```
[1] "\\xff\\xd8\\xff\\xe0\\x00\\x10JFIF\\x00\\x01\\x01\\x00\\x00..."
[1] ff d8 ff e0 00 10 4a 46 49 46 00 01 01 00 00
```

Sometimes we still need to move binary data through a **text-only** channel (e.g. a network, an email, a JSON field).

Base64 encoding

Base64 is another encoding, also lossless. Its job is to turn binary bytes into safe text.

- Each collection of 6 bits is encoded with one character
- So we need $2^6 = 64$ characters for Base64 encoding

```
library("openssl")  
ryan.b64 <- base64_encode(read_file_raw(ryan.jpg.file)[1:15])  
ryan.b64
```

```
[1] "/9j/4AAQSkZJRgABAQAA"
```

Base64 has a cost

This *increases* the storage space required for your data

- Each Base64 character represents six bits, but requires 8 bits to store (UTF-8), so Base64-encoded data is 33% larger!

A (very silly!) example:

```
charToRaw("LSE")          # three bytes (in UTF-8)
encoded <- base64_encode("LSE")
encoded
charToRaw(encoded)         # four bytes (in UTF-8)
```

```
[1] 4c 53 45
```

```
[1] "TFNF"
```

```
[1] 54 46 4e 46
```

Audio data

Continuous data, finite bits

Analog data is **continuous**:

- any value, with infinite precision
- a real sound wave has no gaps

Bits give only a finite menu of patterns (2^n). So:

- A computer can never store raw analog data exactly
- To get it in, we **sample** (measure at chosen moments) and round each value
- This is **digitising**, and it always loses some information

Aside: data with a finite *description* (e.g. $\sin(t)$) *can* be stored exactly. Real signals rarely have one, so we sample.

The basics of sound

Human ears are auditory organs that sense **sound** by detecting fluctuations in air pressure.

The “data” the brain processes (from each ear) is a one-dimensional time series with wave-like structure, a **waveform**.

The link between sound waves and human hearing is complex.

→ For example: how the brain combines data from each ear to interpret a single “sound”

We'll focus on two key properties: **pitch** and **loudness**.

→ **Frequency** (**wavelength**) → pitch

→ **Amplitude** → loudness

The basics of sound

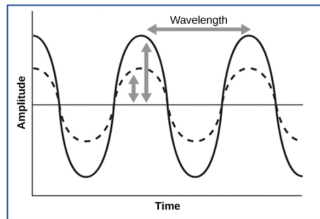


Figure 1. For sound waves, wavelength corresponds to pitch. Amplitude of the wave corresponds to volume. The sound wave shown with a dashed line is softer in volume than the sound wave shown with a solid line. (credit: NIH)

Source: <https://books.psychstat.org/rdata/audio-data.html>

Sound pitch can be measured using frequency in Hertz (Hz).

- If a sound wave completes one cycle in one second = 1 Hz
- Humans can hear sounds from 20 Hz to 20,000 Hz

Loudness can be measured in decibels (dB).

- A conversation is around 60 dB
- Hearing loss can occur with prolonged exposure above 80 dB

Sound digitisation

Sound (as it exists in air) is a time-continuous **analog signal**.

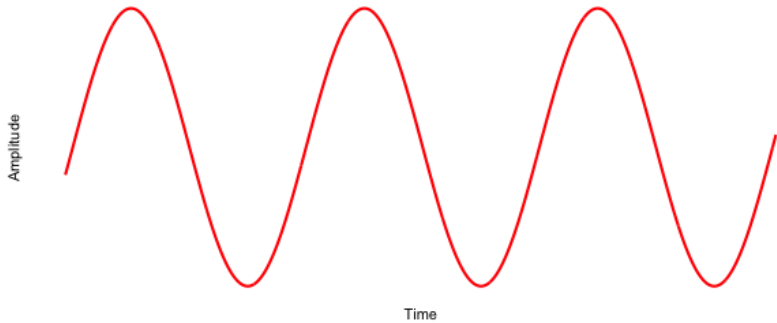
But computer-readable data is represented discretely.

- Digitising sound involves discretising it
- This process is known as **pulse-code modulation (PCM)**, which results in a **PCM stream**

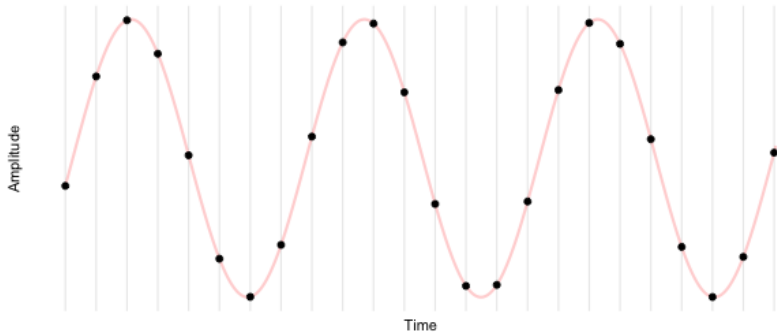
Digitisation happens via **sampling** (not in the statistical sense):

- **Sampling frequency**: samples per second, measured in Hz
- **Quantising**: how are amplitude readings “rounded”?
 - Digitised sound is quantised in **bit depth**, so 4-bit digitised sound measures amplitude in $2^4 = 16$ intervals
- Sampling causes information loss and lowers quality

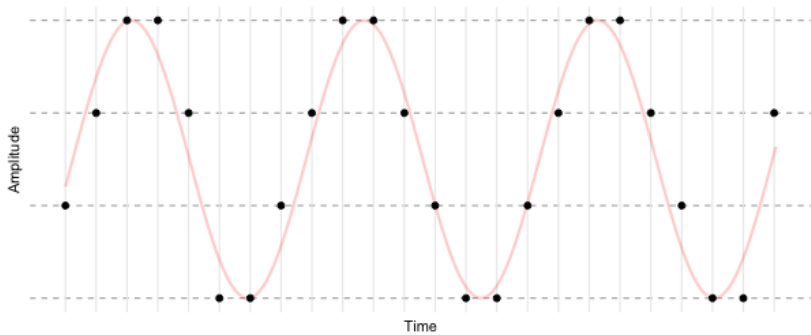
Digital audio sampling



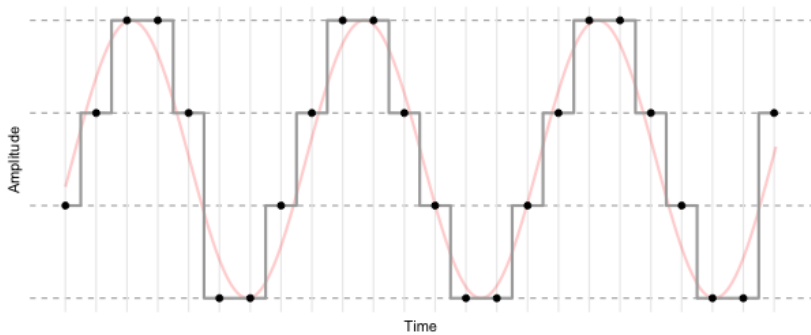
Digital audio sampling



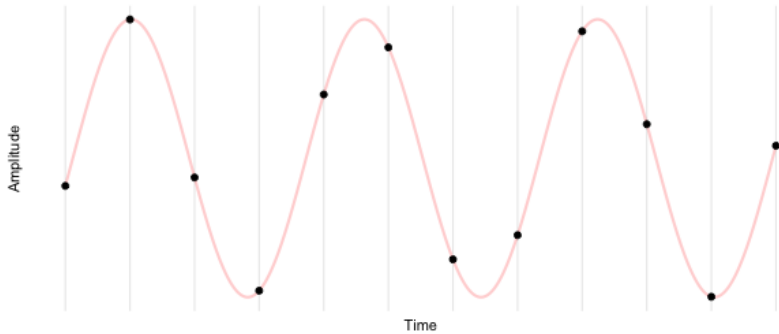
Digital audio sampling



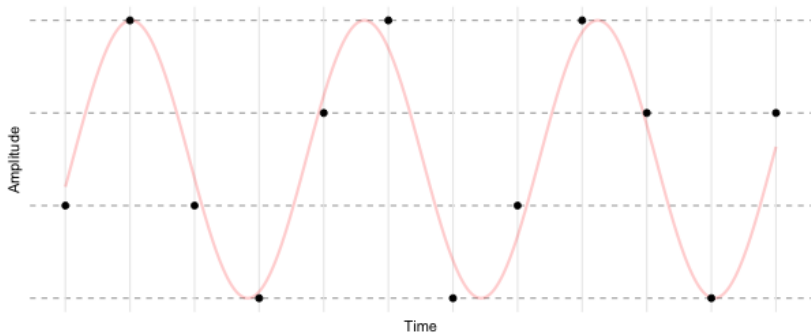
Digital audio sampling



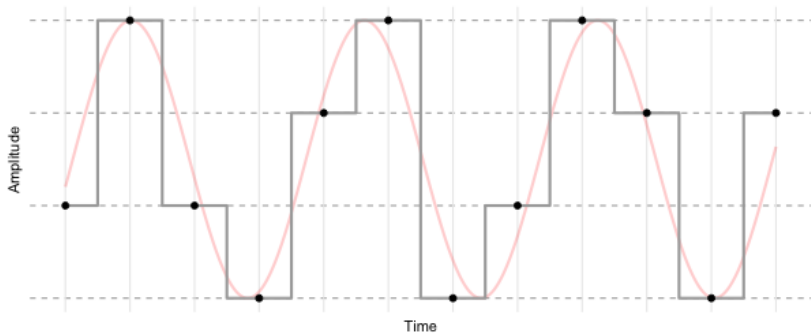
Digital audio sampling



Digital audio sampling



Digital audio sampling



Modern digital audio sampling

“Standard”:

- Sample rate of 44,100 Hz (44.1 kHz)
- Bit depth: 16-bit

High resolution (usually for music):

- Apple Music: up to 24-bit/192 kHz ([source](#))
- Spotify: up to 24-bit/44.1 kHz ([source](#))

For audio on video streaming:

- Usually a sampling rate of 48 kHz ([source](#))

For telephones: can use much lower resolution sampling.

- Sampling rate of 8 kHz, bit depth of 8 bits ([source](#))

Information loss in digitisation

Digitisation involves “fundamental” information loss.

- Analog: infinite data; digital: finite data
- (Side note: the brain also samples from an analog signal!)

Even after digitisation: often too much data to store/transmit.

Digital audio data is often **compressed**: some data is dropped.

- **Lossy compression**: drop data humans can't hear (roughly)
 - Based on psychoacoustic models, depends on audio type
 - Irreversible data loss
- **Lossless compression**: uses prediction models and entropy coding to reduce file size, so no data is lost
 - Useful analogy: UTF-8 is lossless compression of UTF-32 (as we saw with encoding)

Digital audio encoding

Compressing a PCM stream is also called encoding. Unlike character encoding, it can be lossy.

- A **codec** does the encoding and decoding (see **huge list**)
- Lossless codecs decode back exactly; lossy codecs only *approximate* the original

Common lossless codecs:

- **Apple Lossless Audio Codec (ALAC)**: used by Apple Music
- **Free Lossless Audio Codec (FLAC)**: used by Spotify

Common lossy codecs:

- **MP3**: somewhat outdated now
- **Advanced Audio Coding (AAC)**: used by Apple Music, Spotify web player
- **E-AC3 (Dolby Digital Plus)**: used by streaming video apps

Digital audio file types

Digital audio is typically stored in file types called **containers**.

- Container files contain the audio data plus some metadata
- Audio data can be compressed or not
- Many other file types (e.g., video) are also containers

Most common file types for uncompressed data are:

- **Waveform Audio File Format (WAV)**, stored in .wav files
- **Audio Interchange File Format (AIFF)**, stored in .aiff files

Digital audio file types

For compressed data, common file types are:

- MP3-encoded (lossy) data is usually stored in `.mp3`
- AAC-encoded (lossy) data can be stored in `.m4a`, `.mp4`, `.aac`
- E-AC3-encoded (lossy) data can be stored in `.mp4`
- ALAC-encoded (lossless) data can be stored in `.m4a` or `.mp4`
- FLAC-encoded (lossless) data can be stored in `.flac`

Note: file type $\neq$ encoding!

Audio data in R

For audio data in R: `tuneR` and `seewave`.

- Audio data is a sequence of recorded amplitude readings plus sampling information (sampling frequency and bits)
- In `tuneR`, data is loaded as a `Wave` object
 - `@samp.rate` gives the sampling frequency; `@bit` gives the bit depth
 - Access amplitude readings via `@left` and `@right`
 - If mono, there will only be readings in `@left`

Side note: **mono** versus **stereo** refers to how many “channels” are in digitised audio: mono is 1 channel, stereo is 2 channels.

- Many stereo speakers/headphones will play mono audio through both speakers

Audio data in R

MLK's "I have a dream" speech.

→ <https://ia801605.us.archive.org/25/items/MLKDream/MLKDream.mp3>

```
library("tuneR")  
mlk <- readMP3("MLKDream.mp3")  
print(mlk)
```

Wave Object

Number of Samples:	21684288
Duration (seconds):	983.41
Samplingrate (Hertz):	22050
Channels (Mono/Stereo):	Stereo
PCM (integer format):	TRUE
Bit (8/16/24/32/64):	16

Audio data in R

```
str(mlk)
```

```
Formal class 'Wave' [package "tuneR"] with 6 slots
 ..@ left      : int [1:21684288] 0 0 0 0 0 0 0 0 0 0 0 ...
 ..@ right     : int [1:21684288] 0 0 0 0 0 0 0 0 0 0 0 ...
 ..@ stereo    : logi TRUE
 ..@ samp.rate: num 22050
 ..@ bit       : num 16
 ..@ pcm       : logi TRUE
```

```
mlk@left[1000:1010]
```

```
[1] -24 -18  -8  -5 -15 -20 -18 -17 -19 -14  -8
```

Visualising audio data

Since sound is generated by waves, the “data” of sound allows you to (approximately) reconstruct the waves.

→ Remember: sampling + quantising + compression = info loss!

But we have a dimensionality problem:

- time
- amplitude
- frequency

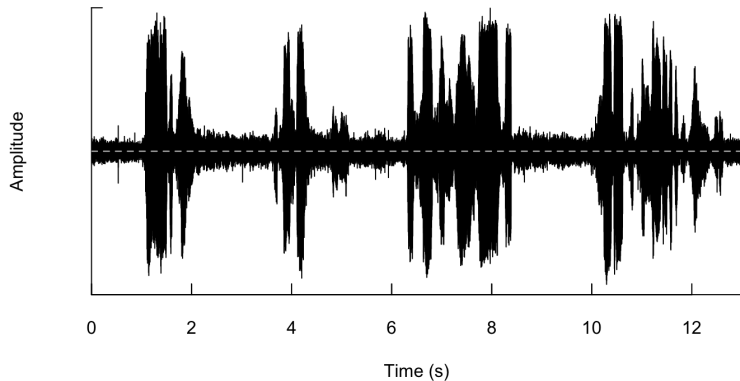
Three basic types of visualisations are useful: **oscillogram**, **periodogram** and a **spectrogram**.

→ Use the seewave package to visualise

Visualising audio data

1. An **oscillogram** is simply a plot of a digitised sound wave.

→ But: a little difficult to “see” the frequency

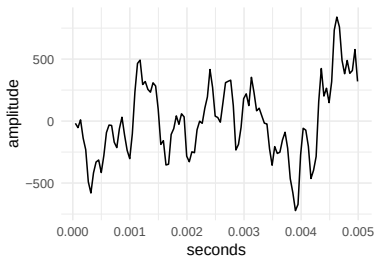


Visualising audio data

You can also plot oscillograms manually using `ggplot2`.

→ Below is an example of a (very) small clip: 110 samples

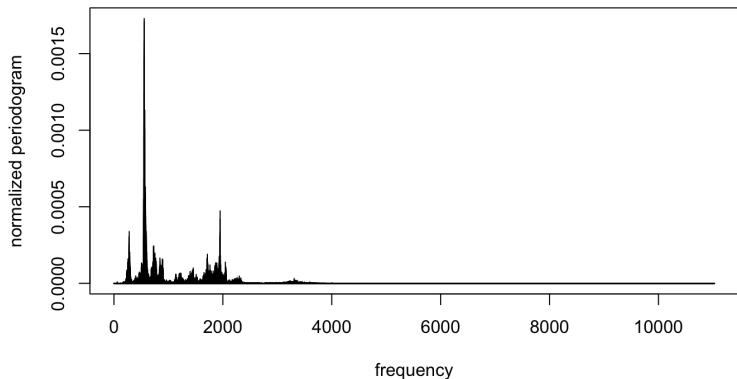
```
mlk.samp <- mlk@left[1:110]  
tp <- tibble(seconds = (1:length(mlk.samp))/mlk@samp.rate,  
              amplitude = mlk.samp)
```



Visualising audio data

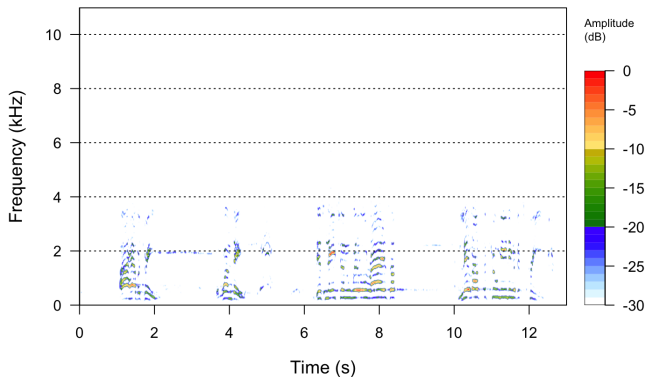
2. A **periodogram** shows the “dominant” frequencies in a recording, i.e. frequencies that contribute most to amplitude.

→ But: it doesn't show time



Visualising audio data

3. A **spectrogram** is a “heat map” that depicts variation in amplitude *and* frequency over time.



Visual data

The basics of vision

Human vision is enabled by “light waves”.

→ **Frequency** (or **wavelength**) determines **colour**

→ Short wavelengths → blue/violet

→ Long wavelengths → red

→ **Amplitude** (or **intensity**) determines **brightness**

→ Brightness $\approx$ how much light energy hits the retina

Unlike sound, light is **electromagnetic**, not mechanical.

→ EM waves oscillate too quickly for eyes to detect the waveform

→ Retinas only detect light intensity

→ Retinas have three types of sensors (“cones”) detecting light intensity across different wavelengths (colours)

Digital images

Digital images are always displayed with **pixels**.

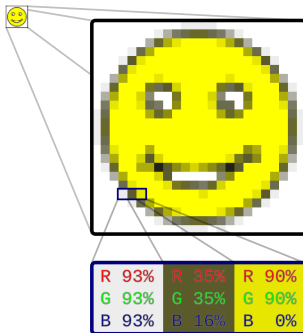
- Small, discrete units arranged in a 2D grid displaying visual data, such as colour and brightness
- E.g., screens contain pixels displaying brightness & colour
- Roughly speaking, human eyes also process data in this way

A pixel grid is a discrete representation of a continuous image that can be stored as bytes.

- Pixels per inch (or cm) is **resolution** (analogous to sampling frequency for audio)
- Colours and brightness stored as discrete values (analogous to quantising for audio)

Pixel grids

A pixel grid up close:



Source: https://en.wikipedia.org/wiki/Raster_graphics

Pixel grids: tessellations

A pixel grid has three components:

1. A grid (or **tessellation**) of equal size cells
2. A single value in each cell of the grid
3. A **lookup table** that maps cell values to colours

A. Cell IDs

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

B. Cell values

92	55	48	21
58	70	NA	37
NA	12	94	11
36	83	4	88

C. Colored values



Source: <https://r.geocompx.org/spatial-class>

Storing images 1: raster

But display $\neq$ storage: two main ways to store digital images.

Raster (or bitmap) storage: store pixel grid directly, values in cells.

→ Many digital images: .tif, .jpg, .gif, .png

Most raster images use the **RGB colour space**.

→ Each raster cell's display colour is determined by a mixture of the three primary colours: red, green and blue

A raster image using the RGB colour space consists of **channels**.

→ A channel is an array storing one layer of pixel data

→ So, for RGB colour space:

→ Each channel is a colour—red, green and blue

→ Can have an additional “alpha” channel (transparency)

→ The resulting image layers channels to generate a rich visual

Storing images 1: raster



Storing images 1: raster



Storing images 1: raster



Storing images 2: vector

Vector graphics store images as geometric objects, such as points, line segments, polygons, etc.

→ Common file types: .svg, .pdf, and .eps

Vector files still display as pixels: the geometries are rasterised on screen.



Raster
GIF, JPEG, PNG



Vector
SVG

Source: https://en.wikipedia.org/wiki/Vector_graphics

Storing images 2: vector

Vector graphics are defined by **geometries**: points, line segments, polygons, curves, etc.

Roughly speaking: vector graphics “trace” images with geometries.

Polygons are a common way to make shapes, defined by:

- **Edges**: line segments
- **Nodes/vertices**: where line segments meet



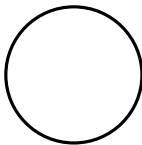
Storing images 2: vector

Vector graphics are often *exact* representations:

- Shapes with a formula (like circles, curves) can be stored exactly using the function.
- Many “real-world” shapes of interest (maps) have no formula

So, we trace them with **vertex samples**.

A (silly) example of vertex sampling, with better examples below.



continuous shape



10 samples



8 samples

Use cases

Raster graphics: images with continuous tones/colours, and/or when efficiency/size is important (e.g., websites).

- “Real-life” photos and videos
- Maps with aerial and sensor imaging
- Topographic maps

Vector graphics: images with geometric shapes and no continuous tones/colours, and/or when rescaling is important.

- Maps of borders and other geographical shapes
- Cartoons, clip art, logos, etc.
- Typography (e.g., PDFs)

Digital photo data

Digital cameras have sensors that measure brightness (luminance) information.

- Brightness information is quantised by bit depth
- Data from sensors combine to form pixels

A **monochrome camera** captures only brightness.

- Generates black and white (technically **greyscale**) images

Sensors in a **colour camera** have filters for red, green and blue.

- The sensors still capture brightness, but for a specific colour
- Each pixel contains data on brightness for each colour

Digital photos are stored as raster data (usually compressed).

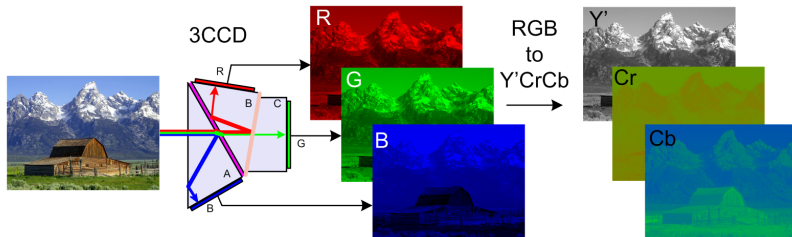
Digital photo compression

JPEG is a common standard for encoding photo data.

These are (roughly) the steps for JPEG encoding:

1. **Convert**: RGB channels to YCbCr channels (*lossless*)
2. **Subsample**: Chroma subsampling (*lossy*)
3. **Transform**: Discrete cosine transform (DCT) applied to blocks of pixels across each channel, yielding frequency coefficients (*lossless*)
4. **Quantise**: Resulting frequency coefficients are quantised, where bit depth varies across frequencies based on a model of human vision (*lossy*)
5. **Compress**: Further *lossless* compression

Digital photo compression



Source: <https://en.wikipedia.org/wiki/YCbCr#/media/File:CCD.png>

Digital photo compression

Y



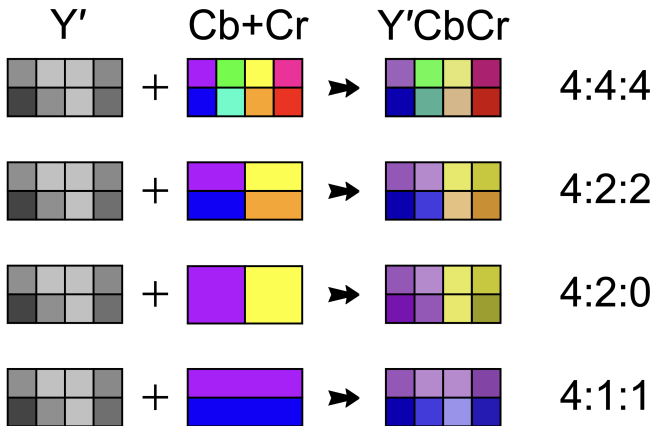
Cb



Cr



Digital photo compression



Source: https://en.wikipedia.org/wiki/Chroma_subsampling

Digital photo quality

Digital photos can be low quality for many reasons.

→ Core issue: information loss during capture or storage

Three major sources:

1. Excessively low pixel density or blur (happens during capture or file resize)
2. Chroma subsampling (happens during encoding)
3. Quantising (happens during capture and encoding)

Some of these are deliberate, some are not!

Digital photo quality

Raster graphics with low pixel density are **pixelated**.

→ “Too few” cells in the grid to represent the image well



Source: https://en.wikipedia.org/wiki/Big_Ben

Digital photo quality

When saving a JPEG, you often have to select “quality”.

→ That setting changes the quantising

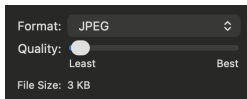
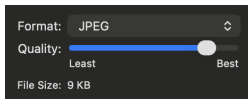


Photo data in R

There are multiple ways to import .jpg photo data into R, but we will use jpeg and exifr.

→ exifr: extract a photo's metadata

→ jpeg: import a .jpg photo and work with its raster data

```
library("exifr")
metadata <- read_exif(ryan.jpg.file)
names(metadata)
```

[1]	"SourceFile"	"ExifToolVersion"
[3]	"FileName"	"Directory"
[5]	"FileSize"	"FileModifyDate"
[7]	"FileAccessDate"	"FileInodeChangeDate"
[9]	"FilePermissions"	"FileType"
[11]	"FileTypeExtension"	"MimeType"
[13]	"JFIFVersion"	"ResolutionUnit"
[15]	"XResolution"	"YResolution"
[17]	"ImageWidth"	"ImageHeight"
[19]	"EncodingProcess"	"BitsPerSample"
[21]	"ColorComponents"	"YCbCrSubSampling"
[23]	"ImageSize"	"Megapixels"

Photo data in R

Reading a .jpg into R with `jpeg` decodes the JPEG file and yields an array object in R.

- A collection of three matrices
- Each matrix is a raster grid corresponding to one of the three colour channels (RGB)
- Each cell in each matrix is a value from 0 to 1, the proportions of red, green and blue, respectively

When image data is “plotted”:

- Each cell in each matrix is quantised to one byte ($2^8 = 256$ values)
- Each pixel’s “full” colour is determined by mixing quantised red, green and blue values

Photo data in R

```
library("jpeg")  
my.img <- readJPEG(ryan.jpg.file)  
str(my.img)
```

```
num [1:200, 1:200, 1:3] 0.345 0.341 0.333 0.322 0.31 ...
```

```
dim(my.img)
```

```
[1] 200 200 3
```

```
# Top corner of the red channel  
my.img[1:3, 1:3, 1]
```

	[,1]	[,2]	[,3]
[1,]	0.3450980	0.3372549	0.3176471
[2,]	0.3411765	0.3333333	0.3137255
[3,]	0.3333333	0.3215686	0.3019608

Photo data in R

```
# Pixel data for the top left pixel
tl.red   <- my.img[1, 1, 1]
tl.green <- my.img[1, 1, 2]
tl.blue  <- my.img[1, 1, 3]
print(c(tl.red, tl.green, tl.blue))
```

```
[1] 0.3450980 0.4392157 0.2431373
```

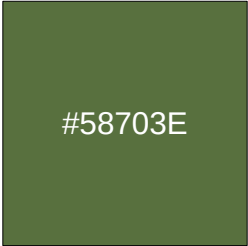
```
library("scales")
tl.pixel <- rgb(tl.red, tl.green, tl.blue)
tl.pixel
```

```
[1] "#58703E"
```

This is a three byte representation of the pixel's colour: 58, 70 and 3e

Photo data in R

```
show_col(tl.pixel, cex_label = 5)
```



#58703E

Photo data in R

```
H <- dim(my.img)[1]
W <- dim(my.img)[2]
tp <- tibble(expand.grid(y = 1:H, x = 1:W))
tp$r <- as.vector(my.img[, , 1])
tp$g <- as.vector(my.img[, , 2])
tp$b <- as.vector(my.img[, , 3])
tp$fill <- rgb(tp$r, tp$g, tp$b)
head(tp)
```

```
# A tibble: 6 x 6
```

	y	x	r	g	b	fill
	<int>	<int>	<dbl>	<dbl>	<dbl>	<chr>
1	1	1	0.345	0.439	0.243	#58703E
2	2	1	0.341	0.435	0.239	#576F3D
3	3	1	0.333	0.427	0.231	#556D3B
4	4	1	0.322	0.416	0.220	#526A38
5	5	1	0.310	0.404	0.216	#4F6737
6	6	1	0.306	0.4	0.212	#4E6636

Photo data in R

```
ggplot(tp, aes(x = x, y = y, fill = fill)) +  
  geom_raster() +  
  scale_fill_identity() +  
  scale_y_reverse() +  
  coord_equal(ratio = 1) +  
  theme_void()
```



Other raster data file types

- Tag Image File Format (TIFF, .tif and .tiff)
 - Common file type for “raw” scanned data
 - Typically lossless compression (or uncompressed)
 - Very large files, not suitable for the web
 - Technically a container: can use .tif for lossy compressed images
- Graphics Interchange Format (GIF): .gif
 - The “ASCII of graphics”: only encodes 256 colours
 - Lossless compression (but still limited by palette)
- Portable Network Graphics (PNG): .png
 - Full colour (but limited by bit depth)
 - Good for screenshots, text, diagrams, logos, line art
 - Not good for very large photos

Spatial data

What is spatial data?

Spatial data is data relating to physical space.

Digitising space runs into the same problem: space is continuous but digital data is discrete.

Social science spatial data is mostly about Earth.

- We will focus on 2D representations of Earth—maps.
- These are just images: either raster or vector!
- So, digital maps involve sampling, which is lossy.

But spatial data raises two distinct issues:

1. Earth may be continuous, but it's not *smooth*. (Bad for math.)
 - Commit to a **datum**: a smooth 3D model of Earth.
2. Space is 3D but maps are 2D.
 - Commit to a **projection** that transforms the 3D model to 2D

Datums and projections also lossy!

Map projections

Mercator Projection
(without Antarctica)



Robinson Projection
(without Antarctica)



Coordinate reference systems

Coordinate reference system (CRS): a way to link coordinates with real places.

- A CRS is a datum + a coordinate system; projection optional.
- **Geographic CRS** (no projection): coordinates are *angles* (longitude/latitude, in degrees).
- **Projected CRS**: coordinates are *distances* (x and y in metres) from an origin.

You need to *choose* a projection that fits your use case.

There are trade-offs, since projections always distort some mix of directions, distances and areas.

Commonly used CRS:

- **WGS 84**: geographic. The datum behind GPS and the coordinates in most web applications (e.g., Google Maps).
- **British National Grid (BNG)**: projected, used in the UK.

Two types of spatial data: raster and vector



Two types of spatial data: raster and vector



Both types involve sampling

Raster and vector maps involve sampling

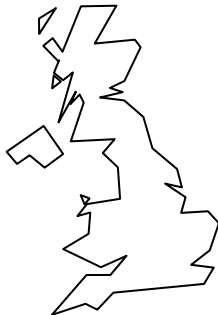
The coastline *looks* smooth, but it's just sampled vertices!

- The real-life coastline has detail at every scale: measured length grows as your ruler shrinks (the **coastline paradox**)

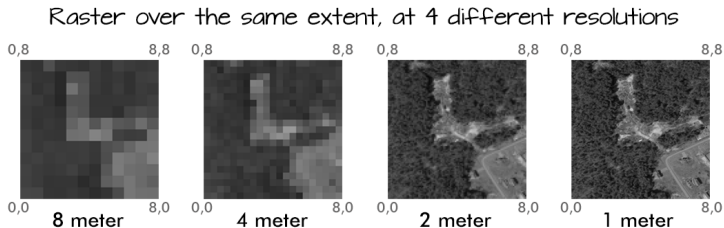
898 vertices



74 vertices



Both types involve sampling



Source: <https://carpentries-incubator.github.io/geospatial-python/instructor/01-intro-raster-data.html>

Spatial data in R

For vector data in R: `sf`.

- Defines spatial data with **simple features** that represent geometric data types + projection.
- Spatial data is loaded into an “sf data frame” with:
 - rows: geographic entities (countries, regions, etc.)
 - columns: relevant “metadata” about each geographic entity plus one column with geometries
 - the geometry column contains vector data for plotting
- The coordinates used for plotting are “scaled” by the projection.

Many ways to store vector data, but a commonly used one is “shapefiles” (`.shp`), which is what we will use.

Spatial data in R

For raster data in R: terra.

- Defines spatial data using a raster grid + projection.
- Data is loaded as a SpatRaster object that is its own class:
 - dimensions: how many cells in the grid (the “resolution”)
 - matrix: stores the values of cells in the raster grid
- The cell locations in the raster grid are “scaled” by the projection.

Raster spatial data is stored as raster graphics, usually in .tif file format.

Mapping in R

You can read a shapefile into R using `read_sf()` from `{sf}`

```
# Read shapefile data into R using
lf <- read_sf("data/ESRI/London_Borough_Excluding_MHW.shp")

# Look at the shape of the data
lf |> select(NAME, geometry) |> head()
```

Simple feature collection with 6 features and 1 field

Geometry type: MULTIPOLYGON

Dimension: XY

Bounding box: xmin: 507007.4 ymin: 155850.8 xmax: 561957.5 ymax: 194889.3

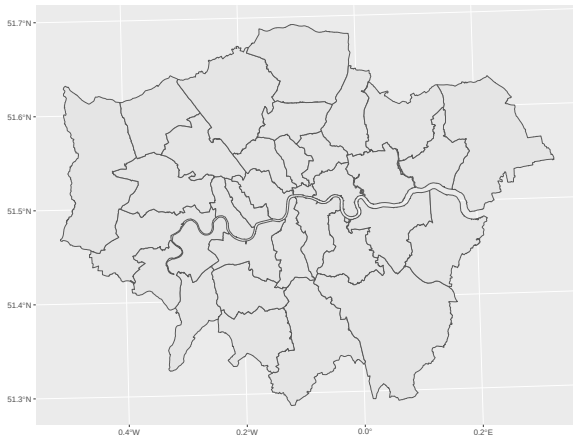
Projected CRS: OSGB36 / British National Grid

A tibble: 6 x 2

NAME	geometry
<chr>	<MULTIPOLYGON [m]>
1 Kingston upon Thames	(((516401.6 160201.8, 516407.3 16021~
2 Croydon	(((535009.2 159504.7, 535005.5 15950~
3 Bromley	(((540373.6 157530.4, 540361.2 15755~
4 Hounslow	(((521975.8 178100, 521967.7 178096.~
5 Ealing	(((510253.5 182881.6, 510249.9 18288~
6 Havering	(((549893.9 181459.8, 549894.6 18146~

Mapping in R: initialise and show borders

```
ggplot(lf) +  
  geom_sf() # optional here: aes(geometry = geometry)
```



Mapping in R

```
lf |>  
  filter(str_detect(NAME, "(Westminster|Camden)")) |>  
  ggplot() +  
  geom_sf(colour = "black", fill = NA, linewidth = 0.5) +  
  theme_void()
```



Mapping in R

You can add points to a map using latitude and longitude

→ Find coordinates using Google Maps

```
pf <- tibble(name = c("LSE", "Buckingham Palace"),
             lon = c(-0.1165, -0.1419),
             lat = c(51.5146, 51.5022)) |>
  st_as_sf(coords = c("lon", "lat"), crs = 4326) |>
  st_transform(crs = 27700) # optional, but smart
print(pf)
```

Simple feature collection with 2 features and 1 field

Geometry type: POINT

Dimension: XY

Bounding box: xmin: 529065 ymin: 179776.9 xmax: 530792.3 ymax: 181200.9

Projected CRS: OSGB36 / British National Grid

A tibble: 2 x 2

	name	geometry
*	<chr>	<POINT [m]>
1	LSE	(530792.3 181200.9)
2	Buckingham Palace	(529065 179776.9)

Mapping in R

```
lf |>
  filter(str_detect(NAME, "(Westminster|Camden)")) |>
  ggplot() +
  geom_sf(colour = "black",
          fill = NA,
          linewidth = 0.5) +
  geom_sf(data = pf,
          colour = "red",
          size = 5) +
  geom_sf_text(data=pf,
              aes(label=name,
                  hjust = c(1,0.7),
                  vjust = c(-1,-1)),
              size=8) +
  theme_void()
```

Mapping in R



Wrong rule, wrong world: spatial mojibake

Remember mojibake: bytes read with the wrong character encoding turn to gibberish.

Coordinates read with the wrong CRS do the same, with geography.

- The landmark coordinates are WGS 84 (degrees); the borough map is BNG (metres)
- Declare the wrong CRS and degrees get read as metres: LSE lands in the sea southwest of Cornwall, near the BNG origin
- Same lesson as encoding: both sides must share the rule

Wrong rule, wrong world: spatial mojibake

The same two numbers, $(-0.1165, 51.5146)$, read with two rules:



✗ LSE, degrees read as metres

Mapping in R

```
lf |>
  ggplot() +
  geom_sf(colour = "gray70",
          fill = "gray95",
          linewidth = 0.5) +
  geom_sf(data = pf,
          colour = "red",
          size = 6) +
  geom_sf_text(data=pf,
              aes(label=name,
                  hjust = c(0.5,1.05),
                  vjust = c(-0.8,0.4)),
              size=10) +
  theme_void()
```

Mapping in R

